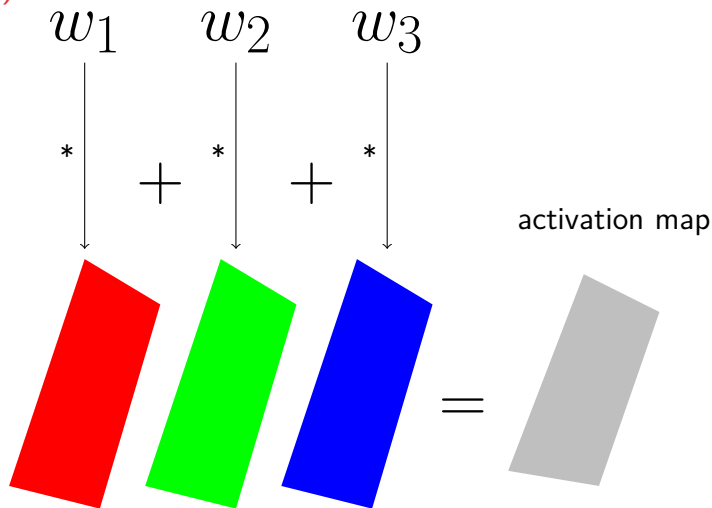


Revision

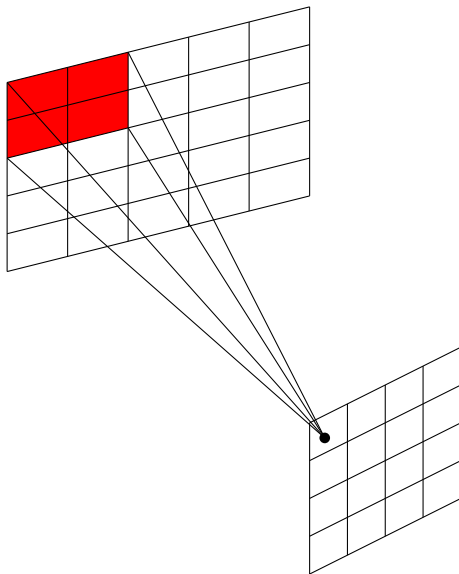
COMP6252 (Deep Learning Technologies)

ECS, University of Southampton

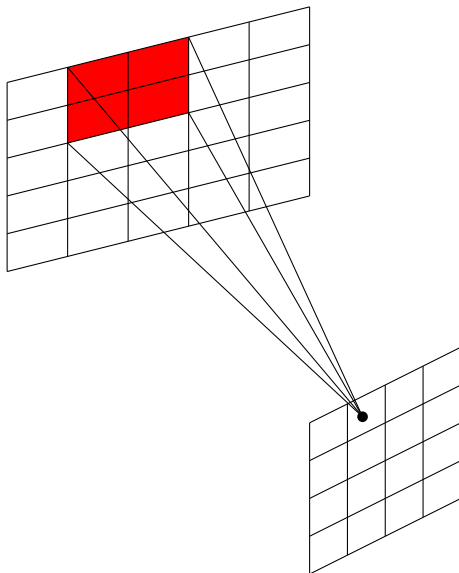
Convolution operation on an input image with 3 channels (RGB)



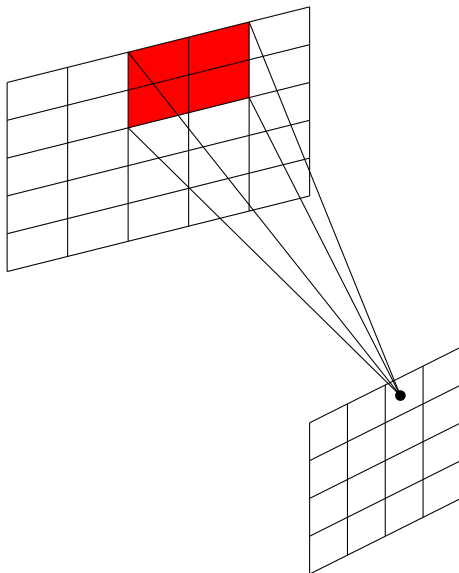
Conv operation on one channel



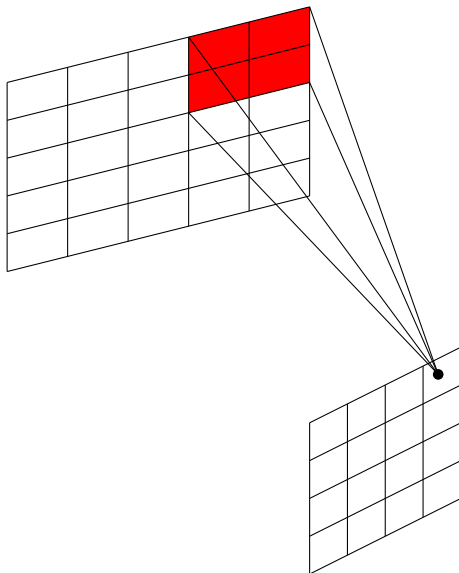
Conv operation on one channel



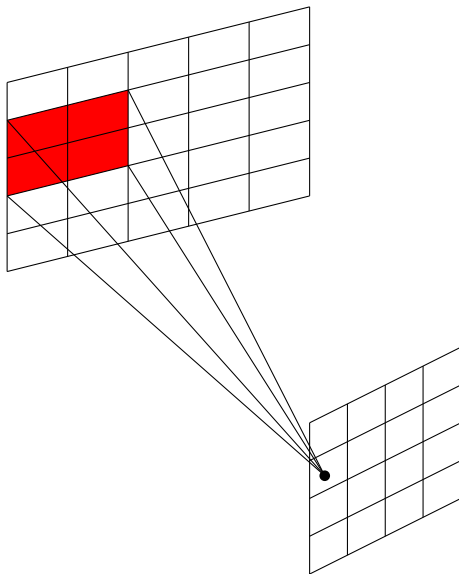
Conv operation on one channel



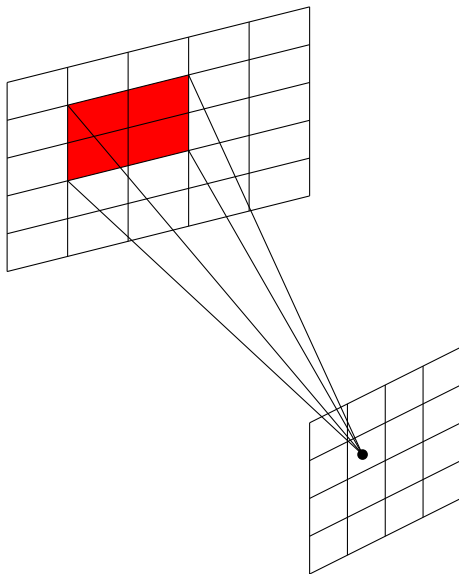
Conv operation on one channel



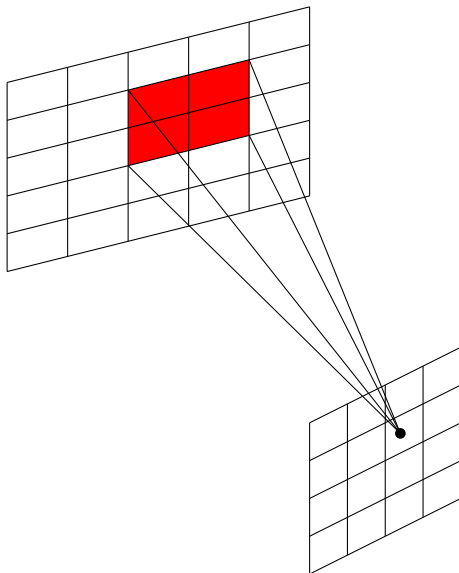
Conv operation on one channel



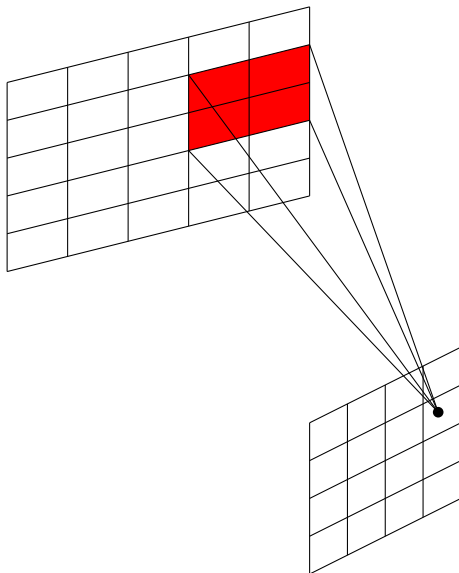
Conv operation on one channel



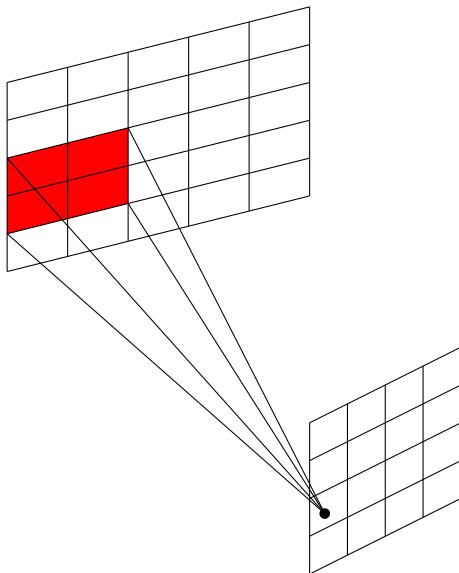
Conv operation on one channel



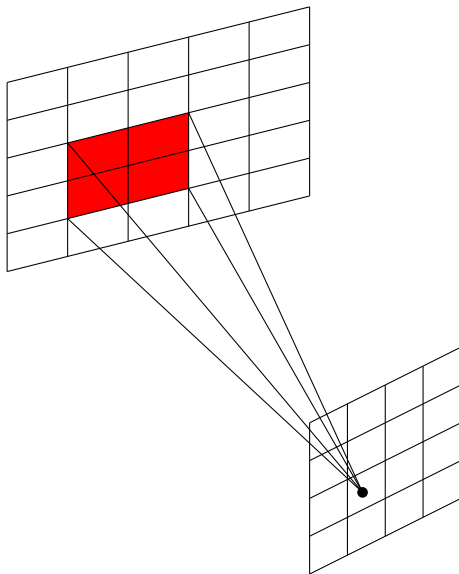
Conv operation on one channel



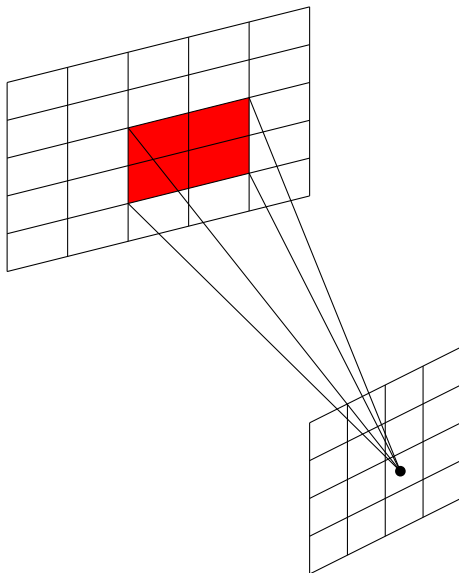
Conv operation on one channel



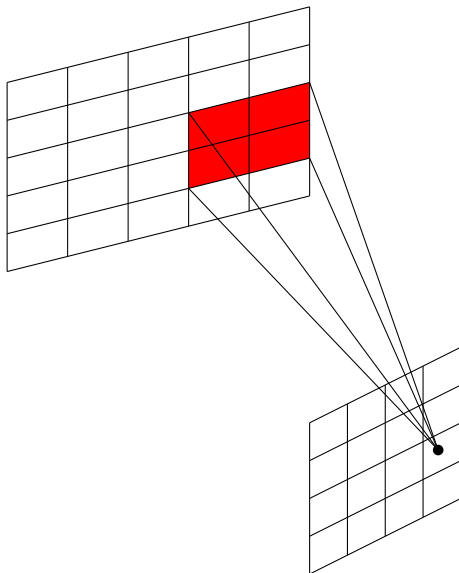
Conv operation on one channel



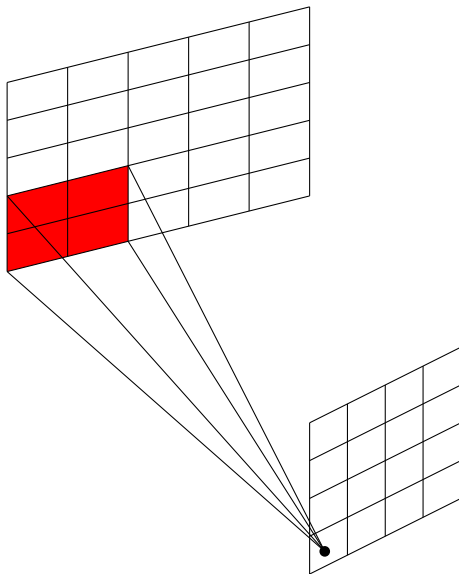
Conv operation on one channel



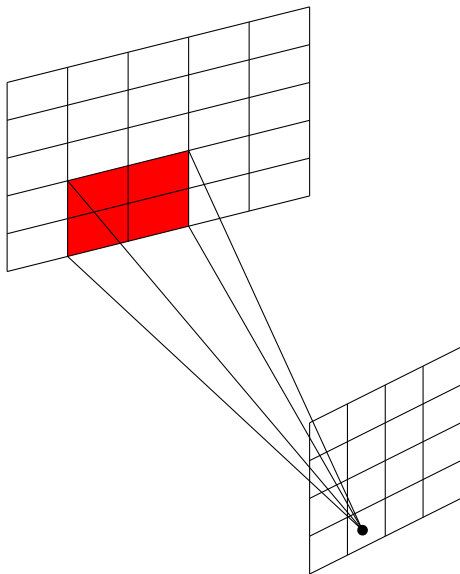
Conv operation on one channel



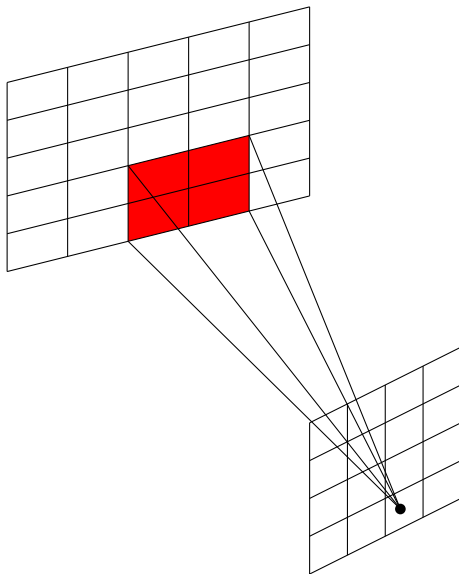
Conv operation on one channel



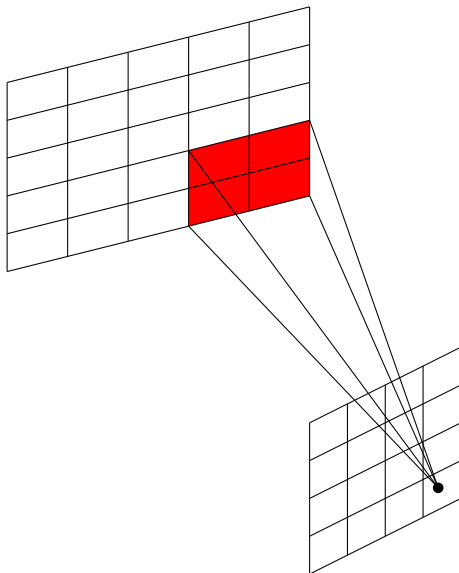
Conv operation on one channel



Conv operation on one channel



Conv operation on one channel



Example: Conv. operation on one channel

Input

1	2	4	3
5	6	8	7
9	10	12	11
13	14	16	2

Kernel

1	-2
-3	4

$\odot$

activation map

=	6		

Example: Conv. operation on one channel

Input

1	2	4	3
5	6	8	7
9	10	12	11
13	14	16	2

Kernel

1	-2
-3	4

$\odot$

activation map

=	6	8	

Example: Conv. operation on one channel

Input

1	2	4	3
5	6	8	7
9	10	12	11
13	14	16	2

Kernel

1	-2
-3	4

$\odot$

activation map

=	6	8	2

Example: Conv. operation on one channel

Input

1	2	4	3
5	6	8	7
9	10	12	11
13	14	16	2

Kernel

1	-2
-3	4

$\odot$

activation map

=	6	8	2
	6		

Example: Conv. operation on one channel

Input

1	2	4	3
5	6	8	7
9	10	12	11
13	14	16	2

Kernel

1	-2
-3	4

$\odot$

activation map

=	6	8	2
	6	8	

Example: Conv. operation on one channel

Input

1	2	4	3
5	6	8	7
9	10	12	11
13	14	16	2

⊙

Kernel

1	-2
-3	4

activation map

=	6	8	2
	6	8	2

Example: Conv. operation on one channel

Input

1	2	4	3
5	6	8	7
9	10	12	11
13	14	16	2

Kernel

1	-2
-3	4

$\odot$

activation map

=

6	8	2
6	8	2
6		

Example: Conv. operation on one channel

Input

1	2	4	3
5	6	8	7
9	10	12	11
13	14	16	2

$\odot$

Kernel

1	-2
-3	4

activation map

=

6	8	2
6	8	2
6	8	

Example: Conv. operation on one channel

Input

1	2	4	3
5	6	8	7
9	10	12	11
13	14	16	2

$\odot$

Kernel

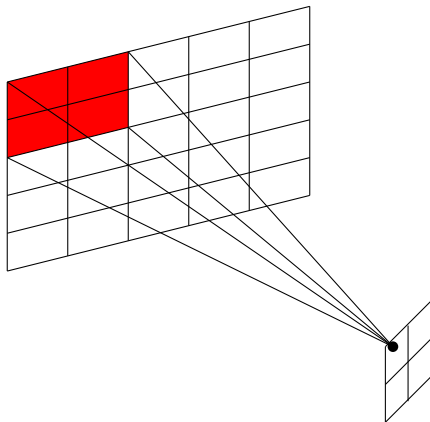
1	-2
-3	4

activation map

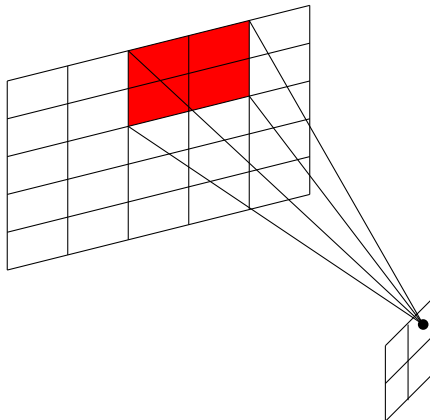
=

6	8	2
6	8	2
6	8	-50

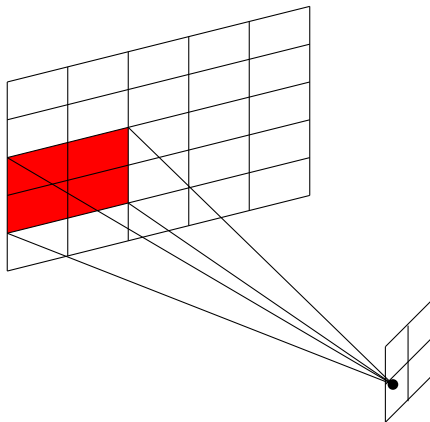
Example with stride=2



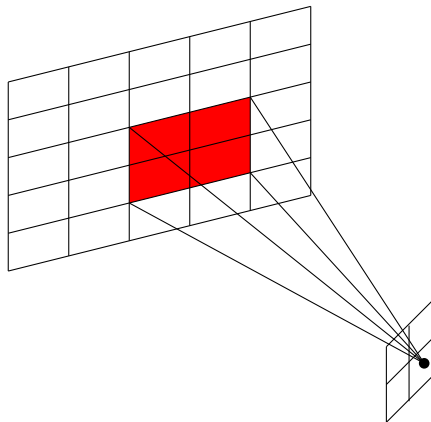
Example with stride=2



Example with stride=2



Example with stride=2



Stride and receptive field

- In the previous example we used a **stride** equal to 1
- The kernel **receptive field** was equal to 2×2
- The size of the output (activation map) was 3×3
- Are those numbers the same in all applications? No

Stride and receptive field

- In general one can use a stride of any size S . Stride of size 1 is the most common.
- Let $F \times F$ be the kernel **receptive field**
- Let $H_i \times W_i \times C_i$ be the size of the input
- The size of the resulting activation map is $H_o \times W_o$ where

$$H_o = \left\lfloor \frac{H_i - F}{S} \right\rfloor + 1$$

$$W_o = \left\lfloor \frac{W_i - F}{S} \right\rfloor + 1$$

Example

- Consider an input image of size (3,28,28)
- We are using the PyTorch convention with the channel dimension first.
- What would be the size of the activation map if a kernel of size 3x3 and stride 2 were used?
- Height and width are the same and equal to $\left\lfloor \frac{28-3}{2} \right\rfloor + 1 = 13$
- So the activation map has size 13x13
- Note that the output of **one** kernel is a 2-d object: the activation map.
- The number of output channels is determined by the **number** of kernels

Parameters

- Consider an image of size $(3, 28, 28)$ used as input to 32 filters of size 3×3
- Since the input has 3 channels each filter has $3 \times 3 \times 3 + 1(\text{bias}) = 28$ Parameters
- With a stride of 1 the output has size $(32, 26, 26)$, i.e., the layer has $32 \times 26 \times 26 = 21632$ nodes
- Total number of parameters $= 32 \times 28 = 896$
- Contrast the above with a feedforward with the first layer having $32 \times 26 \times 26 = 21632$ nodes
- It would have $(3 \times 28 \times 28) \times 21632 = 50878464$ parameters
!(without bias)

Convolution operation mathematically

- Let f, i, j be the filter index, output height index, and output width respectively
- For a stride of 1 (most common), and bias per filter b_f
- The convolution operation is defined as

$$O_{f,i,j} = b_f + \sum_c \sum_{m,n} X_{c,i+m,j+n} * K_{f,c,m,n}$$

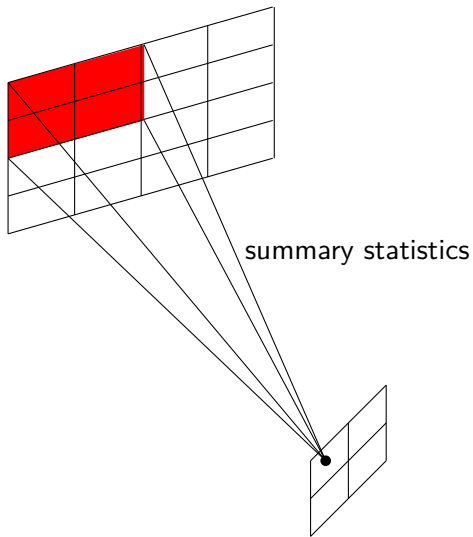
- If one includes the sample index s , needed in the case of batches

$$O_{s,f,i,j} = b_f + \sum_c \sum_{m,n} X_{s,c,i+m,j+n} * K_{f,c,m,n}$$

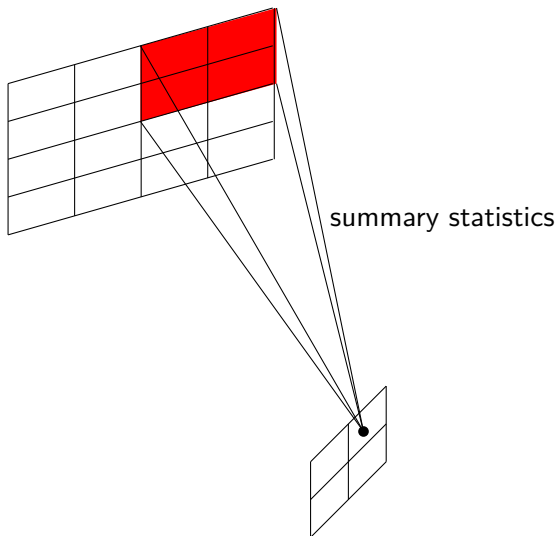
Pooling

- Typically, convolutional networks performs three steps
 - 1 Convolution operation
 - 2 followed by a Non-linear activation such as ReLU
 - 3 followed by **pooling**
- Pooling computes a summary statistics for a small area of the result
- Taking the max value is the most common pooling operation.

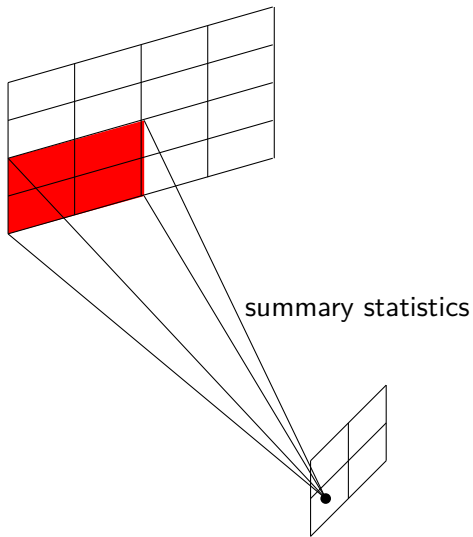
Pooling size 2x2, stride=2



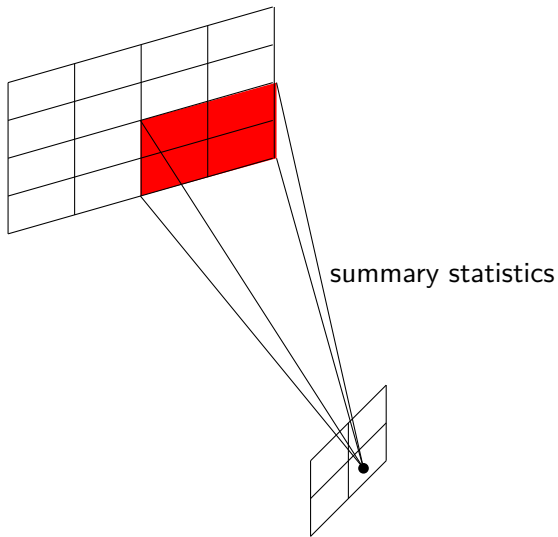
Pooling size 2x2, stride=2



Pooling size 2x2, stride=2



Pooling size 2x2, stride=2



Max pooling

Input

1	2	4	3
5	6	8	7
9	10	12	11
13	14	16	2

Max Pool=

Result

6	

Max pooling

Input

1	2	4	3
5	6	8	7
9	10	12	11
13	14	16	2

Max Pool=

Result

6	8

Max pooling

Input

1	2	4	3
5	6	8	7
9	10	12	11
13	14	16	2

Max Pool=

Result

6	8
14	

Max pooling

Input

1	2	4	3
5	6	8	7
9	10	12	11
13	14	16	2

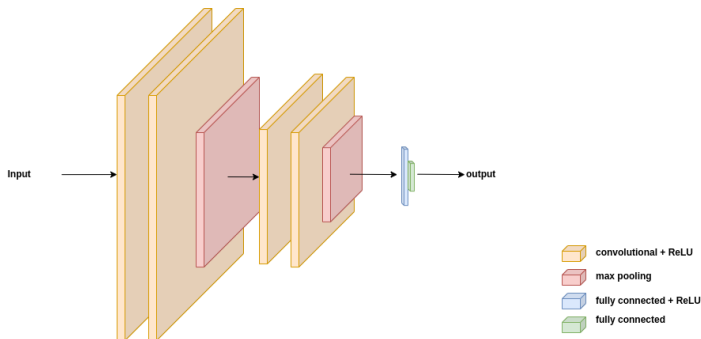
Max Pool=

Result

6	8
14	16

Typical CNN

- Typically, CNN will have multiple blocks
- Each block has multiple Conv layers with ReLU activation
- At the end of a block a max pooling layer



Invariance and Equivariance

- Convolution is a translation equivariant operation
 - It commutes with the translation operation.
 - Let C and S be the convolution and shift operations respectively.
 - For any input x we have

$$C(S(x)) = S(C(x))$$

- By contrast, an operation T is invariant to translation iff

$$T(S(x)) = T(x)$$

- Clearly, convolution is **not** translation invariant
- Max pooling makes the network approximately invariant to small translations

Dropout

- Dropout is another method used to minimize overfitting
- A dropout layer "drops" certain nodes from the NN as shown in the figure below

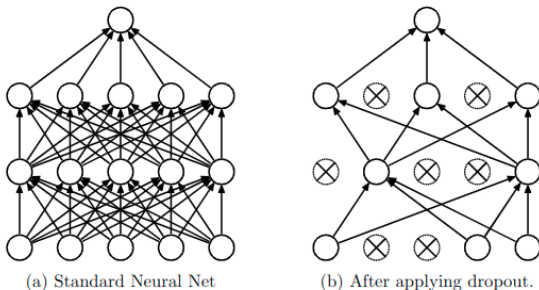


Figure: Srivastava et al.

What is happening exactly?

- Every training pass **different** nodes are dropped
- A neural network with n nodes that uses dropout, can be seen as a set of 2^n different "smaller" networks. Why?
- It means that using dropout is equivalent to averaging over an ensemble of Neural networks sharing the same weights
- Since the noise learned by each "smaller" network is random, "averaging" makes them cancel each other.

Implementation

- Using layers with an on/off switch on nodes is not practical
- More importantly, with an NN with large number of nodes it is **impossible**
- Instead, a node is turned off by multiplying its output with zero **with probability** p
- Mathematically, let $\mathbf{y}^l$ be the output of layer l in a fully connected network
- Then the output of layer $(l + 1)$ is given by ($*$ is element wise product)

$$\mathbf{r}^l = \text{Bernoulli}(p)$$

$$\tilde{\mathbf{y}}^l = \mathbf{r}^l * \mathbf{y}^l$$

$$\mathbf{z}^{l+1} = \mathbf{W}^{l+1} \tilde{\mathbf{y}}^l + \mathbf{b}^{l+1}$$

$$\mathbf{y}^{l+1} = \sigma(\mathbf{z}^{l+1})$$

What about testing?

- We have seen that training a NN with dropout is equivalent to training multiple "smaller" networks.
- The result is an average over all such networks.
- To be consistent we must use the weights learned in the training phase for testing
- This means we have to **average** over all possible networks, which is infeasible.
- We approximate this averaging by **not using dropout** during testing
- But the weights obtained in the test/validate phase are multiplied by the probability p
- Equivalently the weights are multiplied by $\frac{1}{1-p}$ during training

Batch Normalization

- A Batch normalization layer transforms the distribution of the input to a distribution that has 0 mean and unit variance
- Batch normalization is a powerful method that
 - 1 Allows for better convergence of NN training
 - 2 Adding BN layers to existing NN yield better generalization results
 - 3 Reduces over fitting
 - 4 Training is less sensitive to meta-parameters (learning rate, weights initialization)

Batch Normalization- How does it work

- Given a mini-batch of tensors x_{ci} of dimension (S,C,H,W)
- where c is the channel index and i collectively refers to all other dimensions.
- Let $N = S \times H \times W$.
- Batch normalization computes the mean and variance of the batch (per channel) according to

$$\mu_c = \frac{1}{N} \sum_{i=1}^N x_{ci}$$

$$\sigma_c^2 = \frac{1}{N} \sum_{i=1}^N (x_{ci} - \mu_c)^2$$

Batch Normalization- How does it work

The normalized inputs are computed as follows:

$$\hat{x}_{ci} = \frac{x_{ic} - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}}$$

Therefore, for each channel, the $\hat{x}_{ci}$ have zero mean and unit variance. The output of the batch normalization layer is given by

$$y_{ic} = \gamma \hat{x}_{ic} + \beta$$

Where γ and β are **learnable** parameters.

Example

Consider a mini-batch of size two, containing the tensors of size 2×2 , A and B with both having a single channel.

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

- The mean is 4.5 and (biased) variance is 5.25 therefore the output is

$$A = \begin{bmatrix} (1-4.5)/5.25 & (2-4.5)/5.25 \\ (3-4.5)/5.25 & (4-4.5)/5.25 \end{bmatrix}$$

$$B = \begin{bmatrix} (5-4.5)/5.25 & (6-4.5)/5.25 \\ (7-4.5)/5.25 & (8-4.5)/5.25 \end{bmatrix}$$

- When the input has multiple channels, the same operation is performed for each channel independently

Network Architecture

